

Pre-Compiled Most Important AI Libraries, Information and All About CUDA Archs : Flash Attention, xFormers

SECourses: FLUX, Tutorials, Guides, Resources, Training, Scripts
May 24

1 2 1

Our Patreon exclusive posts index list all of the apps we have (over 100+ AI apps, scripts, trainers, presets and more). Use CTRL+F to find whatever app you looking on our index.

Join discord to get help, chat, discuss and also tell me your discord username to get your special rank : SECourses Discord

Please also Star, Watch and Fork our Stable Diffusion & Generative AI GitHub repository and join our Reddit subreddit and follow me on LinkedIn (my real profile)

=======

- **Pre-compiled Libraries Online Links : All_Compiled_Libraries_Online_Install_Links.txt**
- **You can download individual compiled whl files from very bottom attachments section**

Pre-Compiled Libraries Information and Importance

- When it comes to installing and setting up AI repositories and apps, some libraries are so hard to install
- Unless you use pre-compiled libraries, they will take massive time to compile while installing or fail in most cases
- Moreover, finding accurately pre-compiled libraries is a massive problem as well
- Because, when you pre-compile a CUDA depended library, you have to include accurate CUDA archs to support GPUs and use accurate Torch, CUDA and Python

Skip navigation

versions

- Therefore, I will start publishing my pre-compiled libraries in this post
- Recently for ACESTEP 1.5 XL Training I have compiled Flash Attention and I am starting this post with it : https://www.patreon.com/posts/157675060
- If you don't use accurately compiled library with your system setup, it will not work and you will get errors
- On Windows, if your pre-compiled library size becomes too big, sadly it doesn't work therefore I will exclude server GPUs from Windows compiles

1 June 2026 Update

- xformers added to the inventory

Our So Far Libraries List - Starting From 23 May 2026

- To understand CUDA Arch versions and which GPUs they cover look below CUDA Archs table
- flash_attn-2.8.4+cu130torch211cuda131sm86sm89sm120-cp311-cp311-win_amd64.whl (23 May 2026 - took over 7 hours on AMD 9 - 9950X CPU) :
  - Windows, Python 3.11.x, Torch 2.11, CUDA 13.x, CUDA Archs 8.6;8.9;12.0
    - **8.6:** A40, A10, A16, A2, RTX A6000, RTX A5000, RTX A4000, RTX A3000, RTX A2000, RTX 3090 Ti, RTX 3090, RTX 3080 Ti, RTX 3080, RTX 3070 Ti, RTX 3070, RTX 3060 Ti, RTX 3060, RTX 3050 Ti, RTX 3050
    - **8.9:** L4, L40, L40S, RTX 6000 Ada, RTX 5000 Ada, RTX 4500 Ada, RTX 4000 Ada, RTX 4000 SFF Ada, RTX 2000 Ada, RTX 4090, RTX 4080, RTX 4070 Ti, RTX 4070, RTX 4060 Ti, RTX 4060, RTX 4050
    - **12.0:** RTX PRO 6000 Blackwell Server Edition, RTX PRO 4500 Blackwell Server Edition, RTX PRO 6000 Blackwell Workstation Edition, RTX PRO 6000 Blackwell Max-Q Workstation Edition, RTX PRO 5000 Blackwell, RTX PRO 4500 Blackwell, RTX PRO 4000 Blackwell, RTX PRO 4000 Blackwell SFF Edition, RTX PRO 2000 Blackwell, RTX 5090, RTX 5080, RTX 5070 Ti, RTX 5070, RTX 5060 Ti, RTX 5060, RTX 5050
- flash_attn-2.8.4+cu130torch211cuda131trimarch-cp311-cp311-linux_x86_64.whl (23 May 2026 - spent like 50$ on RunPod for fast compile) :
  - Linux, Python 3.11.x, Torch 2.11, CUDA 13.x, CUDA Archs 8.0;8.6;8.9;9.0;10.0;10.3;12.0
    - Same as above plus server GPUs
    - **8.0:** A100, A30
    - **9.0:** GH200, H200, H100
    - **10.0:** GB200, B200
    - **10.3:** GB300, B300
- xformers-0.0.35+03b91d7d.d20260601-py39-none-linux_x86_64.whl and xformers-0.0.35+03b91d7d.d20260601-py39-none-win_amd64.whl (1 June 2026)
  - Python 3.9-3.10-3.11-3.12-3.13, Torch 2.10-2.11, CUDA 13.x, CUDA Archs 8.0;8.6;8.7;8.9;9.0;9.0a;10.0;10.0a;10.3;10.3a;11.0;11.0a;12.0;12.0a;12.1;12.1a+PTX
- cumesh-0.0.1-1torch2.9.1cuda13allarchs-cp311-cp311-linux_x86_64.whl (28 May 2026) :
  - Linux, Python 3.11.x, Torch 2.9.1, CUDA 13.x, CUDA Archs 7.5;8.0;8.6;8.7;8.8;8.9;9.0;10.0;10.3;11.0;12.0;12.1

Skip navigation

CUDA GPU Architecture List Explained: Compute Capabity, sm_XX, PTX, and the Full GPU Mapping

A practical guide to this CUDA/PyTorch architecture list:

> 8.0;8.6;8.7;8.9;9.0;10.0;10.3;11.0;12.0;12.1+PTX

What This Architecture List Means

- This list is a set of **CUDA GPU architecture targets**. It tells a CUDA build system, PyTorch extension build, or similar GPU compiler flow which NVIDIA GPU generations to compile for.
- It is **not** a list of CUDA Toolkit versions. For example, 12.0 in this list means **compute capability 12.0**, not "CUDA 12.0." NVIDIA defines compute capability as the version number that identifies the GPU features, hardware parameters, and supported instructions for a device. NVIDIA also states that a GPU's compute capability maps directly to an SM version: for example, compute capability 12.0 corresponds to sm_120.
- In PyTorch extension builds, a similar list is commonly passed through:

> TORCH_CUDA_ARCH_LIST="8.0 8.6 8.7 8.9 9.0 10.0 10.3 11.0 12.0 12.1+PTX"

- PyTorch documents TORCH_CUDA_ARCH_LIST as the way to explicitly choose which compute capabilities a CUDA extension should support. PyTorch also documents the +PTX suffix, which adds PTX intermediate code for the specified compute capability to improve forward compatibility.

What Is Difference Between 12.0 vs 12.0a or 9.0 vs 9.0a?

- In CUDA, when you see compute capability versions like sm_120 vs sm_120a (or compute_120 vs compute_120a), the difference comes down to the **a suffix**, which stands for **"accelerated"** (sometimes described as architecture-specific) features.

The Core Difference

- **12.0 (e.g., sm_120)**
  This targets the **base feature set** of the architecture. Code compiled for sm_120 is more portable and follows the standard compatibility rules—it can typically run on that architecture and is forward-compatible with later GPUs of the same major family through PTX JIT compilation.
- **12.0a (e.g., sm_120a)**
  The a variant enables **architecture-specific accelerated features** that are exclusive to that exact GPU architecture. These are typically new hardware instructions—often related to **Tensor Cores, specialized matrix multiply (WGMMA/MMA) operations, or memory features**—that NVIDIA exposes only for that specific chip generation.

Key Practical Implications

- The most important tradeoff is **portability versus access to cutting-edge features**:
  - Code compiled with the a suffix is **locked to that specific architecture**. It is generally **not forward-compatible**—you cannot rely on it running on future architectures via PTX JIT, because those accelerated instructions may change or not exist elsewhere.

- The non-a version maintains the normal CUDA compatibility guarantees, so it's the safer default unless you specifically need the accelerated instructions.

Quick Reference

Aspect	`sm_120`	`sm_120a`
Feature set	Base/standard	Accelerated, architecture-specific
Forward compatibility	Yes (via PTX JIT)	No (locked to arch)
Use case	General code	Specialized high-perf kernels

This `a` suffix convention started becoming prominent with `sm_90a` (Hopper), where NVIDIA introduced features like WGMMA that needed the architecture-specific designation, and it has continued with newer architectures.

When You'd Use a

- You typically only need the a variant when writing **high-performance libraries** (like cuBLAS, CUTLASS, or custom kernels) that want to squeeze out maximum performance using the very latest hardware-specific instructions, such as the newest Tensor Core MMA shapes introduced with that architecture.

What Is CUDA Compute Capability?

- Every NVIDIA GPU has a **compute capability**, written as X.Y, such as 8.6, 9.0, or 12.1.
- The number tells CUDA what the GPU can do. It affects things like supported instructions, memory behavior, Tensor Core features, scheduling features, and other architecture-level capabilities. NVIDIA's CUDA Programming Guide says the general specifications and features of a compute device depend on its compute capability.
- The format is simple:

> Compute Capability 8.6
> Major version: 8
> Minor version: 6
> Native binary target: sm_86
> Virtual PTX target: compute_86

- So when you see:

> 8.6

- CUDA build tools usually translate that into targets like:

> sm_86
> compute_86

- NVIDIA's documentation explains that sm_XY is the binary architecture label corresponding to compute capability X.Y, while compute_XY is the PTX virtual architecture target.

sm_XX vs compute_XX: The Important Difference

- CUDA has two major forms of GPU code:

Term	Meaning	Example
`sm_86`	Native GPU binary target, also called a cubin target	Optimized machine code for CC 8.6 GPUs
`compute_86`	PTX virtual architecture target	Intermediate PTX code for CC 8.6+ JIT compilation
Cubin	Compiled binary code for a specific GPU SM version	`sm_80`, `sm_90`, `sm_120`
PTX	Intermediate virtual ISA used by CUDA	`compute_80`, `compute_90`, `compute_121`
Fatbin	Container that can hold multiple cubins and PTX versions	One library can include `sm_80`, `sm_86`, and `compute_121`

- A **cubin** is fast because it is already compiled for a real GPU architecture. PTX is more flexible because the NVIDIA driver can JIT-compile it at runtime for a newer GPU. CUDA applications and libraries often store GPU code inside a **fatbin**, which can contain multiple cubins and PTX entries. At runtime, CUDA picks the best compatible code for the GPU in the machine.

Why +PTX Matters

- Your list ends with:

> 12.1+PTX

- That means: build for compute capability 12.1, and also include PTX for that architecture.
- In PyTorch's documentation, +PTX causes extension binaries to include PTX instructions for the specified compute capability. That PTX can then runtime-compile for GPUs with compute capability greater than or equal to the specified one. PyTorch notes that this improves forward compatibility, but exact native targets are usually better for performance when you know the GPUs you are targeting.
- In practical terms:

> 12.1+PTX

- roughly means:

> native binary: sm_121
> PTX fallback: compute_121

- That is useful because if a future NVIDIA GPU appears with a compatible newer compute capability, the driver may be able to JIT-compile the embedded PTX instead of requiring a rebuild.

Binary Compatibility vs PTX Compatibility

- CUDA compatibility has two important rules.

1. Cubin / sm_XX compatibility

- Native cubin binaries are generally compatible **within the same major compute capability**, as long as the running GPU has an equal or higher minor version.
- For example:

> sm_86 can run on CC 8.6 and CC 8.9
> sm_86 cannot run on CC 8.0
> sm_86 cannot run on CC 9.0

- NVIDIA explains that a cubin compiled for sm_86 can load on compatible sm_8x GPUs with an equal or higher minor version, but it will not load across major compute capability versions such as from 8.x to 9.0.

2. PTX / compute_XX compatibility

- PTX is different. PTX for a given compute capability can be JIT-compiled for GPUs with that compute capability or newer.
- For example:

> compute_80 PTX can JIT for CC 8.0+
> compute_90 PTX can JIT for CC 9.0+
> compute_121 PTX can JIT for CC 12.1+

- NVIDIA's CUDA documentation says PTX can be JIT-compiled for any compute capability equal to or higher than the PTX target, enabling forward compatibility with future GPUs.

Full GPU Architecture Mapping

Arch entry	Native CUDA target	PTX target	Main GPU family	GPUs
8.0	sm_80	compute_80	Ampere data center	A100, A30
8.6	sm_86	compute_86	Ampere	A40, A10, A16, A2, RTX A6000, RTX A5000, RTX A4000, RTX A3000, RTX A2000, RTX 3090 Ti, RTX 3090, RTX 3080 Ti, RTX 3080, RTX 3070 Ti, RTX 3070, RTX 3060 Ti, RTX 3060, RTX 3050 Ti, RTX 3050
8.7	sm_87	compute_87	Jetson Orin	Jetson AGX Orin, Jetson Orin NX, Jetson Orin Nano
8.9	sm_89	compute_89	Ada Lovelace	L4, L40, L40S, RTX 6000 Ada, RTX 5000 Ada, RTX 4500 Ada, RTX 4000 Ada, RTX 4000 SFF Ada, RTX 2000 Ada, RTX 4090, RTX 4080, RTX 4070 Ti, RTX 4070, RTX 4060 Ti, RTX 4060, RTX 4050
9.0	sm_90	compute_90	Hopper	GH200, H200, H100
10.0	sm_100	compute_100	Blackwell data center	GB200, B200
10.3	sm_103	compute_103	Blackwell data center	GB300, B300
11.0	sm_110	compute_110	Jetson T-series	Jetson T5000, Jetson T4000
12.0	sm_120	compute_120	Blackwell RTX / RTX PRO	RTX PRO 6000 Blackwell Server Edition, RTX PRO 4500 Blackwell Server Edition, RTX PRO 6000 Blackwell Workstation Edition, RTX PRO 6000 Blackwell Max-Q Workstation Edition, RTX PRO 5000 Blackwell, RTX PRO 4500 Blackwell, RTX PRO 4000 Blackwell, RTX PRO 4000 Blackwell SFF Edition, RTX PRO 2000 Blackwell, RTX 5090, RTX 5080, RTX 5070 Ti, RTX 5070, RTX 5060 Ti, RTX 5060, RTX 5050
12.1+PTX	sm_121	compute_121 included	IVIDIA GB10	NVIDIA GB10 / DGX Spark

- 8.0 / sm_80 :
  - A100, A30
- 8.6 / sm_86 :
  - A40, A10, A16, A2, RTX A6000, RTX A5000, RTX A4000, RTX A3000, RTX A2000, RTX 3090 Ti, RTX 3090, RTX 3080 Ti, RTX 3080, RTX 3070 Ti, RTX 3070, RTX 3060

Ti, RTX 3060, RTX 3050 Ti, RTX 3050

- 8.7 / sm_87 :
  - Jetson AGX Orin, Jetson Orin NX, Jetson Orin Nano
- 8.9 / sm_89 :
  - L4, L40, L40S, RTX 6000 Ada, RTX 5000 Ada, RTX 4500 Ada, RTX 4000 Ada, RTX 4000 SFF Ada, RTX 2000 Ada, RTX 4090, RTX 4080, RTX 4070 Ti, RTX 4070, RTX 4060 Ti, RTX 4060, RTX 4050
- 9.0 / sm_90 :
  - GH200, H200, H100
- 10.0 / sm_100 :
  - GB200, B200
- 10.3 / sm_103 :
  - GB300, B300
- 11.0 / sm_110 :
  - Jetson T5000, Jetson T4000
- 12.0 / sm_120 :
  - RTX PRO 6000 Blackwell Server Edition, RTX PRO 4500 Blackwell Server Edition, RTX PRO 6000 Blackwell Workstation Edition, RTX PRO 6000 Blackwell Max-Q Workstation Edition, RTX PRO 5000 Blackwell, RTX PRO 4500 Blackwell, RTX PRO 4000 Blackwell, RTX PRO 4000 Blackwell SFF Edition, RTX PRO 2000 Blackwell, RTX 5090, RTX 5080, RTX 5070 Ti, RTX 5070, RTX 5060 Ti, RTX 5060, RTX 5050
- 12.1+PTX / sm_121 :
  - NVIDIA GB10 / DGX Spark

Attachments

Download all

- flash_attn-2.8.4+cu130torch211cuda131sm86sm89sm120-cp311-cp311-win_amd64.whl
- flash_attn-2.8.4+cu130torch211cuda131trimarch-cp311-cp311-linux_x86_64.whl
- cumesh-0.0.1-1torch2.9.1cuda13allarchs-cp311-cp311-linux_x86_64.whl
- All_Compiled_Libraries_Online_Install_Links.txt
- xformers-0.0.35+03b91d7d.d20260601-py39-none-linux_x86_64.whl
- xformers-0.0.35+03b91d7d.d20260601-py39-none-win_amd64.whl

2 comments

Join the conversation...

Sugar Coat VFX Design · 1mo

could you explain little more how I use this lib? For example, when i choose one of the template from Rupod, how I refer to which lib in your list?

1 1

SECourses: FLUX, Tutorials, Guides, Resources, Training, Scripts AUTHOR · 1mo

it is pip install lib_url. easy as that. you can use All_Compiled_Libraries_Online_Install_Links.txt to get urls

Collapse replies

Related posts >



VisoMaster Fusion and VisoMaster Automatic Installer - The Most Advanced 0-Shot Face Swap / Deep Fake APP - State of...

1-Click to install VisoMaster Fusion (newer much better) and classic VisoMaster on Windows and also on Massed Compute - SOTA Face Sw...

4 days ago 19 107



IndexTTS2 SECourses Premium Voice Cloning and Generation App - 1-Click to Install on Windows, RunPod and Massed...

Patreon exclusive posts index to find our scripts easily, Patreon exclusive posts index to see which scripts updated or added last and amazi

May 10 9 77



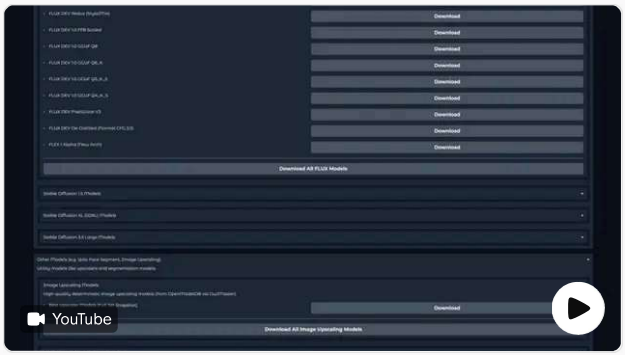
The Pai

Full In th

4 ho

This gain

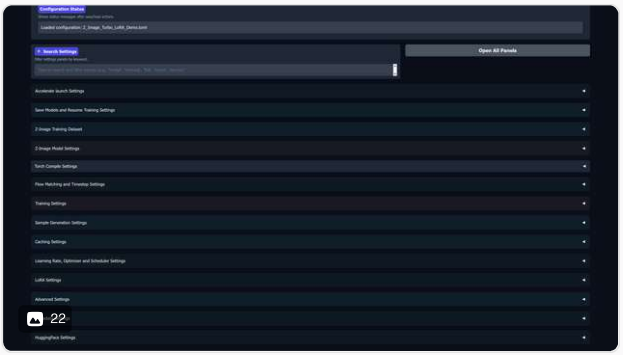
Popular posts >



SwarmUI Auto Installer + The Ultimate Image and Video AI Models Downloader - For Windows, RunPod and Massed...

Unified AI Models Downloader for SwarmUI, ComfyUI, Automatic1111 and Forge Web UI, Forge Web UI Classic and Neo - Supports SD 1.5, SDXL,...

April 26 41 535



SECourses Musubi Tuner - 1-Click to Install App for LoRA Training and Full Fine Tuning Qwen Image, Qwen Image Edit, Wan 2.1 an...

Patreon exclusive posts index to find our scripts easily, Patreon scripts updates history to see which updates arrived to which scripts and

April 20 33 444



Co Ada

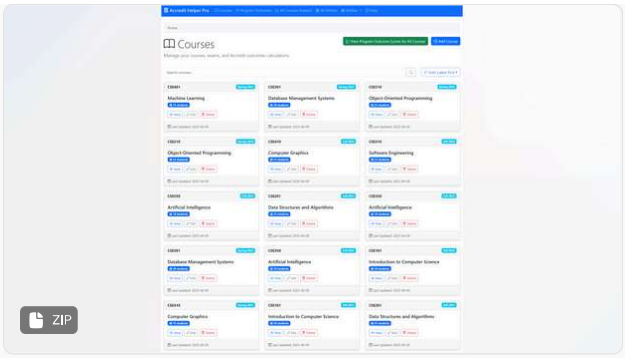
Patr upd

Apr

This gain

This video uses YouTube's privacy-enhanced mode. YouTube may still store and/or gain access to information on your device, including cookies and local storage.

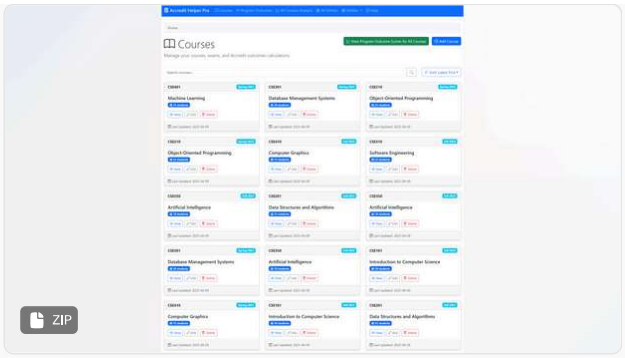
Popular products >



Accredit Helper Pro: University / Institution Wide License - Lifetime

Step by Step Full Tutorial and Guide that shows full features as well : https://www.youtube.com/watch?v=B6E9IF8SkoA This is University /...

$5,000



Accredit Helper Pro: 10x Speed Up Your Accreditation Lifetime License

Step by Step Full Tutorial and Guide that shows full features as well : https://www.youtube.com/watch?v=B6E9IF8SkoA This is Department...

$300

$ 1

$10

$10

SECourses: FLUX, Tutorials, Guides, Resources, Training, Scripts

Gift

Home Posts Quips Collections Shop Membership Recommendations